

Apple Support AI Agent

Hiver SDE Internship Take-Home Assignment

Objective. Build an AI customer-support agent for AppleSupport conversations from the Customer Support on Twitter dataset. The system classifies customer messages, estimates confidence, generates safe replies for low-risk requests, and escalates risky or ambiguous cases.

Executive Summary. The final system uses TF-IDF unigram/bigram features with Logistic Regression for intent classification, followed by a safety-oriented routing layer. After comparing rule-based and embedding approaches, TF-IDF + Logistic Regression was selected. On a 200-example development Golden Set it achieved 68.5% accuracy, 67.9% macro F1, and 67.6% weighted F1. Risk rules override model confidence for sensitive requests.

Evaluation caveat. The Golden Set and training labels were rule-assisted/weakly supervised rather than independently human-annotated. The benchmark may therefore overestimate real-world generalization and should be treated as a development benchmark.

1. Dataset & Intent Taxonomy

The primary source is Thought Vector's *customer-support-on-twitter* dataset. The project extracts AppleSupport conversations and customer messages associated with the AppleSupport account. The raw dataset is excluded from GitHub because of its large size; processed datasets and evaluation artifacts are included.

Intents: ios_update, battery_charging, device_hardware, app_issues, performance, screen_display_keyboard, connectivity, apple_id_icloud, app_store_itunes, apple_music_media, billing_payment_purchase, orders_products_warranty, other.

Exploratory topic analysis was used to identify recurring support themes before defining the taxonomy.

2. Golden Evaluation Set

A 200-example Golden Set was created across the taxonomy. Each record contains the customer message, expected intent, expected action, escalation reason, and review status. Candidate selection and labeling were rule-assisted and intended for development evaluation.

Limitation: this is not independent human ground truth. A stronger future benchmark should use a fresh, independently human-annotated holdout set.

3. Baselines & Model Selection

System	Accuracy	Macro F1
Rule-based baseline	64.0%	58.2%
TF-IDF + Logistic Regression	68.5%	67.9%
Embedding prototype	61.0%	60.6%
Embedding KNN	58.5%	55.6%

Rule baseline. A transparent keyword/rule classifier provided the first benchmark. It reached 64.0% accuracy and 58.2% macro F1.

Final classifier. TF-IDF with unigrams and bigrams feeds Logistic Regression with balanced class weights. The 200 Golden Set examples were excluded from training. Weak supervision produced 38,982 training examples after filtering and class capping.

Embedding experiments. SentenceTransformer prototype and nearest-neighbor approaches were tested but underperformed the TF-IDF classifier.

4. Agent Architecture & Routing

Pipeline: Customer message → text normalization/TF-IDF → Logistic Regression → intent + confidence → risk checks + multi-issue detection + confidence threshold → AUTO_HANDLE or ESCALATE.

Escalation: refund/payment disputes, account compromise/security, legal/serious complaints, hardware repair/replacement/service, multiple distinct issue groups, or confidence below 0.55. Risk rules take priority over confidence.

Replies: low-risk intents use safe intent-specific templates that ask for relevant information without unsupported promises. Billing disputes are escalated instead of promising refunds.

5. Evaluation Results

Intent: 68.5% accuracy, 67.9% macro F1, 67.6% weighted F1. There were 63 intent errors. Confidence ranged from 0.160 to 0.999, with mean approximately 0.698; 48 examples were below the 0.55 threshold.

Routing benchmark: action accuracy 76.5%; escalation F1 25.4%. The agent predicted 145 AUTO_HANDLE and 55 ESCALATE, versus 192 and 8 expected actions in the rule-assisted benchmark. This indicates conservative routing.

Why accuracy can mislead: accuracy hides per-class performance and class imbalance. Macro F1 is more informative because each intent contributes equally. More importantly, the labels are rule-assisted, so the benchmark is not proof of production performance.

6. Failure Analysis

- 1. Billing vs battery charging:** “charge” can mean battery charging or a monetary charge.
 - 2. Device hardware vs connectivity:** device complaints can contain Wi-Fi/Bluetooth/connection language.
 - 3. Device hardware vs performance:** slow, frozen, or malfunctioning devices can resemble hardware faults.
 - 4. Billing vs App Store:** purchase and payment vocabulary overlaps.
 - 5. iOS update vs battery/performance:** customers often report battery drain or performance degradation after updates.
- These errors show that short support tweets can require context beyond isolated keywords.

7. Limitations & Future Improvements

The main limitations are weakly supervised training labels, a rule-assisted Golden Set, short ambiguous tweets, and conservative routing that can over-escalate. Future work should use an independently human-annotated holdout, improve context-aware classification and confidence calibration, add explicit multi-intent modeling, and retain deterministic safety checks around any future retrieval/LLM response generator.

8. Reproducibility & Repository

The repository contains source scripts, processed datasets, evaluation outputs, README, and .gitignore. The raw *twcs.csv* is excluded. The README documents installation and the ordered pipeline. Final demo: `python src/12_support_agent.py`.

GitHub: <https://github.com/harini-here/apple-support-agent>

Conclusion

The project demonstrates an end-to-end support-agent workflow: data extraction, intent design, Golden Set construction, baseline comparison, supervised classification, risk-aware routing, safe response generation, evaluation, and failure analysis. TF-IDF + Logistic Regression was the strongest tested classifier, while the separate escalation layer prevents high-risk requests from being auto-handled solely because the classifier is confident.